models heavily rely on the quality and accessibility of the underlying data. You can use techniques like `VACUUM`, liquid clustering, `OPTIMIZE`, and bucketing to store, access, and manage your data efficiently. Optimized tables ensure that the data feeding into ML algorithms is processed efficiently. We'll cover these briefly here, but also suggest that you refer to the Delta Lake documentation for a comprehensive understanding of each technique.

Vacuum

The `VACUUM` command is crucial in managing resources within Delta tables. It works by cleaning up invalidated files and optimizing the metadata layout. If your Delta table undergoes frequent **Data Manipulation Language** (**DML**) operations (update, insert, delete, merge), we recommend running the `VACUUM` operation periodically. DML operations can generate numerous small files over time. Failure to run `VACUUM` could eventually result in many small files with minimal data remaining in each, leading to poor performance.

Note that `VACUUM` doesn't run automatically; you must explicitly schedule it. Consider scheduling `VACUUM` at regular intervals, such as weekly or monthly, depending on your data ingestion frequency and how often you update the data. Additionally, you have the option to configure the retention period to optimize storage and query performance. The `delta.logRetentionDuration` Delta configuration command allows you to control the retention period by specifying the number of days you want to keep the data files. This means you will delete all transaction log data for a table that goes back beyond the retention setting, e.g., only the last seven days of metadata remain. The transaction log is not very big, but this way you can control the duration the Delta table retains data files before the `VACUUM` operation removes them. Note that the retention duration affects your ability to look back in time to previous versions of a table. Keep that in mind when deciding how long you want to retain the metadata and transaction log.

Liquid Clustering

Liquid Clustering is a great stand-in for partitioning; see how to implement it in *Figure 3.2*. Frequently, data gets over-partitioned, leaving too few files in a partition or unbalanced partitions. You do not need partitioning unless a table is a terabyte or larger. In addition to replacing manual partitioning strategies, liquid clustering also replaces the need for Z-ordering on Delta tables. Z-ordering is not compatible with clustered tables. When choosing columns to cluster on, include high cardinality columns you use for query filters. Another commonly given example is a timestamp column. Instead of creating a derived column date to minimize the cardinality, simply cluster on the timestamp. Liquid clustering benefits Delta tables with skewed data distribution or changing access patterns. It allows the table to adapt to analytical needs by redefining clustering keys without rewriting data, resulting in optimized query performance and a flexible, maintenance-efficient structure. With liquid clustering enabled, you must use DBR 13.3+ to create, write, or `OPTIMIZE` Delta tables. The following figure shows how to put liquid clustering into practice.